



Soccer Infrared - Team KiloHertz

Elettra Robotics Lab (ERL) & IIS L. Cobianchi, Verbania

ITALY

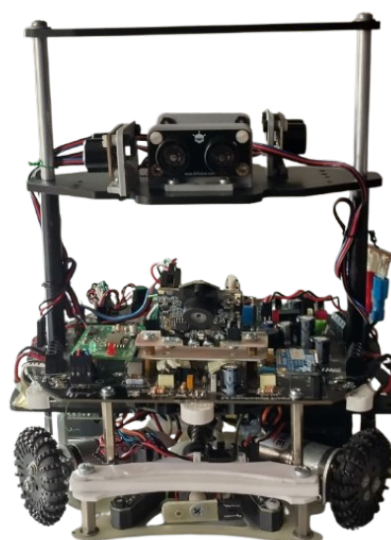
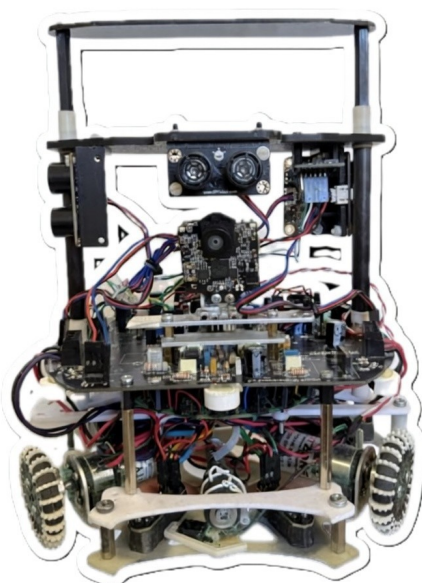
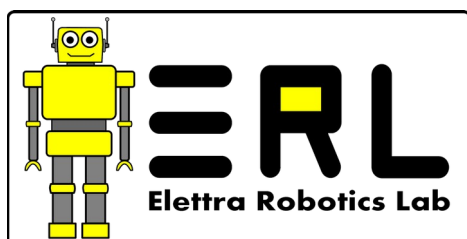


Table of Contents

- 1. Introduction and robot's initial overview**
 - 1.1. *I/O Pinout of **Viper S and S²** main board*
- 2. Robot's general logic and behavior**
 - 2.1. *Attack and defense strategies*
 - 2.2. *Attack maneuvers*
- 3. Electronic hardware and software**
 - 3.1. *Custom PCBs*
 - 3.1.1. Kicker Board
 - 3.1.2. Main Board
 - 3.2. *Sensors and measuring units*
 - 3.2.1. Main MCU
 - 3.2.2. IMU
 - 3.2.3. Pixy Camera
 - 3.2.4. Ultrasonic sensors
 - 3.2.5. Ball Holder Sensor
 - 3.2.6. Custom line sensors
 - 3.2.7. Secondary MCU
 - 3.2.8. Custom Ball Detection Board
 - 3.2.9. Ball following algorithm
- 4. Mechanical and structural hardware**
 - 4.1. *Custom omnidirectional wheels*
 - 4.2. *Chassis*
 - 4.3. *Mouth*
 - 4.4. *kicker*

4.5. Omnidirectional movement

5. Gallery

Team members

- Bogni Francesco
- Jonghi Davide
- Tessarolo Carlo
- Massaro Matteo

Mentors

- Raimondo Sgro'
- Vellone Daniel

Team Introduction

We are the Italian soccer Infrared team **KiloHertz**. Our team consists of four students in their 4th year of high school at the **Lorenzo Cobianchi** Institute, in **Verbania**, 1 following the *electronics systems design* course and other members following the *ICT course*. This experience helped introduce robotics to more students in our school and its competitions.

Our team has been assisted by professor Raimond Sgro' and past member Daniel Vellone whose insight and experience with the past competitions helped us get prepared for **Vienna**.

Our team is organized as follows:

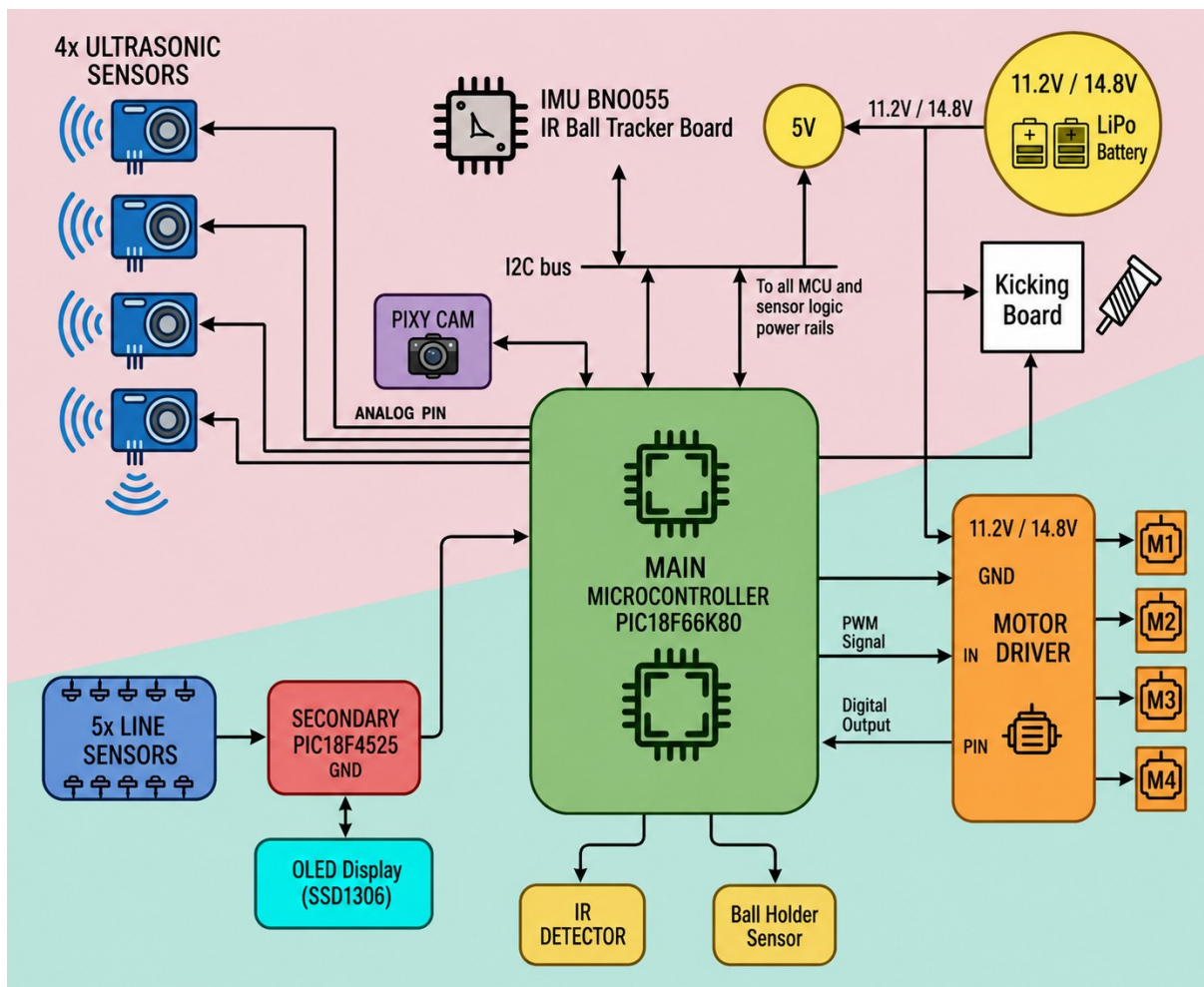
- Jonghi Davide and Tessarolo Carlo : **Main MCU software design.** attack and defense strategies.
- Bogni Francesco : **CAD Design.** Overall robot structure, omni wheel design and production and **Electronics Design.** Custom PCB design, dribbler design and production

-
- Massaro Matteo :**Camera Vision, Hardware supporter**, assembled wheels and takes care of the robot body, documentation and poster.

1. Introduction and robot's initial overview

This year our robots are **Viper S** and **Viper S²**. We participated with them in the **Robocup Junior Academy 2026, Catania**. **S** is the main attacker while **s²** is the "goalkeeper" as he is shorter and more stable, but the roles in the game are dynamic and change depending on the situation.

Structurally speaking, they are the same. The boards used are the same on both robots, this makes it easier on our side since we don't need multiple versions of code for each robot. All of the **Viper S** and **S²** components communicate with each other via different protocols. Here follows a block diagram of the robot's structure.



Viper S and Viper S² Block Diagram

Our robots follow a hierarchy structure where the top of it is embodied by the main MCU : **PIC18F66K80** . This 8 bit MCU runs at *48 MHz* and receives the information using multiple UART ports and also direct connections. The second most important MCU on the main logic board is the **PIC18F4525**, whose purpose is to read the analog custom made line sensors and inform the central unit when a line is crossed through a *Low Priority Interrupt* and then sends which sensor has been activated using one of the 5 direct lines. It's also important to note that the robots use a removable bluetooth module which is used in the robots strategies and debugging purposes using a computer.

Other peripherals that the logic board manages are :

- Four **DFRobot SEN0307**
- One **ball holder sensor**
- One **Oled SSD1306 Display**
- One **BNO055 IMU**
- One **Kicker control board**
- One **Custom Kicker**
- One **Pixy Camera**
- One **Custom Ball detection board**

The main logic board is also populated with plenty of LEDs and a selector used for debug purposes and tactics switching during matches.

1.1. I/O Pinout of Viper S/S² main board

PIN	I/O	Function	PIN	I/O	Function
RA0	AI	ULTR_DX	RA6	-	-
RA1	AI	ULTR_SX	RA7	-	-
RB0	DI	INTLIN (Interrupt Linea)	RD0	DO	M1_A
RB1	DI	LIN_AV_DX	RD1	DO	M1_B

RB2	DI	P_AVVIO (SW1)	RD2	DO	M2_A
RB3	DI	AD_SEL	RD3	DO	M2_B
RB4	DI	ARIETE	RD4	DO	M3_A
RB5	-	-	RD5	DO	M3_B
RB6	DI	LIN_AV	RD6	DO	M4_A
RB7	DI	LIN_IND	RD7	DO	M4_B
RC0	DO	MODULE ROBOCUP	RE0	DO	LED_ON (LED1)
RC1	DO	LANCIO	RE1	DO	LED_PWR (LED2)
RC2	-	-	RE2	DO	LED_UL (LED3)
RC3	-	I2C_SCL	RE3	-	-
RC4	-	I2C_SDA	RE4	DI	P_BUS (SW2)
RC5	DO	BUZZ	RE5	DI	P_AUX
RC6	-	-	RE6	-	-
RC7	-	-	RE7	-	-
RF0	DI	IR0	RG0	-	-
RF1	DI	IR0DX	RG1	DI	LIN_DX
RF2	DI	IR0DX2	RG2	DI	LIN_SX
RF3	DI	IR0_REAR	RG3	-	-
RF4	DI	IR0SX2	RG4	DI	TOCCOPIN (Ball Presence)
RF5	DI	IR0SX	RG5	-	-
RF6	DI	RUN	RG6	-	-
RF7	DI	SEL_AUX	RG7	-	-

2. Robot's general logic and behavior

As we said before the two robots have different roles on the field. While developing their behavior we identified the three main different scenarios that each robot could encounter, then we implemented each robot's "personality" according to their role and to the current scenario they'd be put in.

2.1 Attack and defense strategies

Since each robot is able to communicate to the other using their **bluetooth module** we planned to make them share important information about their state. Since we defined **S²** as the defender and **S** as the striker it would be useful, firstly, to communicate the "**I'm Alive(K)**" character to the other robot. In this way in the event one robot exits out of bounds, thus has to be removed from the field for one minute, the other one stops receiving the character. This triggers a condition which commands the robot to roll back into defense in case it doesn't have sight of the ball despite their role. Just this simple control helps us receive less goals since we have almost always a robot actively defending from opponents scores.

Next the robots share their "**Ball status**" which essentially tells the other one if it has the ball in sight or in possession. With this information the other robot falls back in defense while the one with the ball attempts to score.

Lastly when both robots don't see the ball we have defined their behavior based on the **A/D** (Attack / Defense) **selector**. The robot that is marked as *defender* will place itself in front of the ally goal. Meanwhile the robot who is defined as an *attacker* will drive towards the center of the field, this should give it a higher chance of spotting the ball, thus start to chase it. These maneuvers are performed with the use of distance measures such as ultrasonic and ToF sensors. We also use the camera goals information to center our robot in front of the ally goal.

The **A/D** selector also influences the role communicated to the other robot.

2.2 Attack maneuvers

Some maneuvers that we have implemented for scoring are the following :

- **Standard/Torsion :**

One of the fundamental kinematic strategies developed to optimize the finishing (scoring) phase is the twist maneuver. This maneuver consists of a controlled rotation of the robot around its vertical axis, the amplitude of which angle is dynamically modulated based on the game context.



Rotation Angle Calculation Algorithm

The execution of this maneuver relies on processing data from two distinct sensing systems:

- Ultrasonic sensors
- Target goal localization systems

The kinematics of the maneuver establish a correlation between the robot's proximity to the opponent's back wall and the amplitude of the twist angle: the shorter the distance detected from the target, the larger the yaw angle applied to accurately direct the shot.

Oscillation Mitigation and Signal Conditioning

During the experimental testing phase, a dynamic instability phenomenon was observed, characterized by unwanted oscillations around the rotation axis. This behavior prevented the system from achieving the correct orientation within the required timeframe.

To overcome this control issue, a compensation and filtering algorithm was introduced directly into the sensor data acquisition phase. The raw value read by the transducers is corrected by multiplying it by the cosine of the desired target angle.

The introduction of this trigonometric attenuation factor allows for the proportional dampening of the control action as the robot approaches the target angle, successfully stabilizing the maneuver and eliminating the overshoot effect.

```

LEGGI_ULTRA();
//TORSIONE
if ((DIR < 10 && DIR!=0) && (ULFRONT <= 150) && (ULDX <= 60 || ULSX <= 60) && (IR0 || IR0_REAR || IR0DX || IR0SX || TOCCO == 1)){

// TORX CON ULTRASUONI
    if(ULDX > 55 && ULSX > 55){torsione(0);torx = 0;}
    else if(ULSX <= 55 /*&& ULDX >= 100*/) //accentrarsi a DX
    {
        if (ULFRONT>=90) {torsione(35);torx = 35;}
        else if(ULFRONT >=60){torsione(45);torx = 45;}//TORX=20; TORX=30;
        else{torsione(60);torx = 60;};
        attivoTorx = 1;
    }

    else if(ULDX <= 55 ) //accentrarsi a SX
    {
        if (ULFRONT>=90) {torsione(-35);torx = -35;}
        else if(ULFRONT >=60){torsione(-45);torx = -45;}//TORX=-20; TORX=-30;
        else{torsione(-60);torx = -60;};
        attivoTorx = 1;
    }
    else{torsione(0);torx = 0;};
}
else{torsione(0);torx = 0;};

FINETORX:

if(TORX == 20 || TORX == 30 || TORX == -20 || TORX == -30){
FLAG_TORX = 1;
}
else {FLAG_TORX = 0;}//per debug

```

```

void LEGGI_ULSX (void)
{
    ULSX_MED=0;
    for (U = 0; U < 3; U++)
    {

        ADCON0 =0b00000101;

        ADCON0bits.GO = 1;    // Start conversion
        while (ADCON0bits.GO); // Wait conversion done

        APP_H = ADRESH;
        APP_L = ADRESL;
        ULSX = ((APP_H << 8) + APP_L)/8; // Read ADC registers and convert to 0-1023 integer
        //NB convertitore a 12 bit!!

        ULSX_MED = ULSX+ULSX_MED;
    }
    ULSX = ULSX_MED/3;

    ULSX_2 = ULSX;

    if(TORX >= 30 && TORX<=45)
    {
        ULSX_2 = ULSX_2 * 0.7;
    }
    else if(TORX>=45 && TORX<=60)
    {
        ULSX_2 = ULSX_2 * 0.5;
    }
    ULSX=ULSX_2;
}

```

"Torsion " maneuver's code extract

3. Electronic hardware

The electronic hardware on our robots allow us to perform all kinds of tasks. Without these external reference points it would be nearly impossible for our robots to move in straight lines, follow the ball, decide when to shoot and when to stop before it gets out of bounds. On top of that our custom PCBs house all the microcontrollers and other hardware used by the robot to perform his actions and by us to monitor the state of each algorithm.

3.1. Custom PCBs

Our custom PCBs were all designed using **Kicad 8**, an open source and free EDA software perfect for beginners and more skilled designers. Through its schematic and PCB editor we were able to draw our boards from the ground up without any issues. We found plenty of plugins that aided our development process. Once the boards were ready for production we reached out to a local company named **Millennium Dataware SRL** who offered to produce them and sponsor us. Although we had a stable supplier for our PCB we still had to rely on some 3rd party services and their fast shipping for a batch of boards that wasn't planned. The first version of the kicker board would generate a lot of electrical noise and its capacitor charging speed was low, due to a design flaw in the placement of the components.

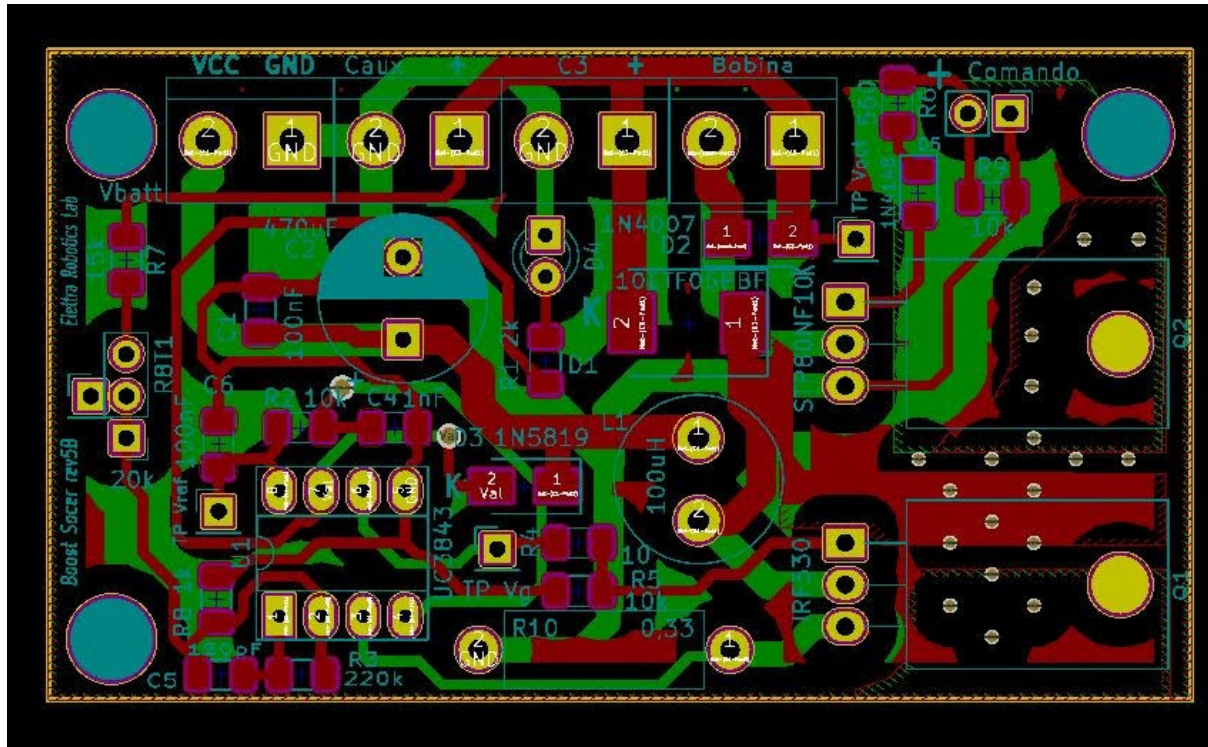
3.1.1 Kicker Board

The **kicker boost board**, also referred to as “**Shot card**” by us, is designed to actuate a “homemade” electromagnetic solenoid. The “**Shot card**” purpose can be split into two main actions :

1. Charge the capacitors at a voltage of 48 V using a step-up circuit and a supply of 12 V or 16 V (3S or 4S lipo battery voltage).

2. Quickly discharge the capacitors in the solenoid when the main MCU requests it through a direct line. Therefore a burst of current passes inside the copper windings which generates a strong magnetic field that accelerates the iron piston inside the solenoid.

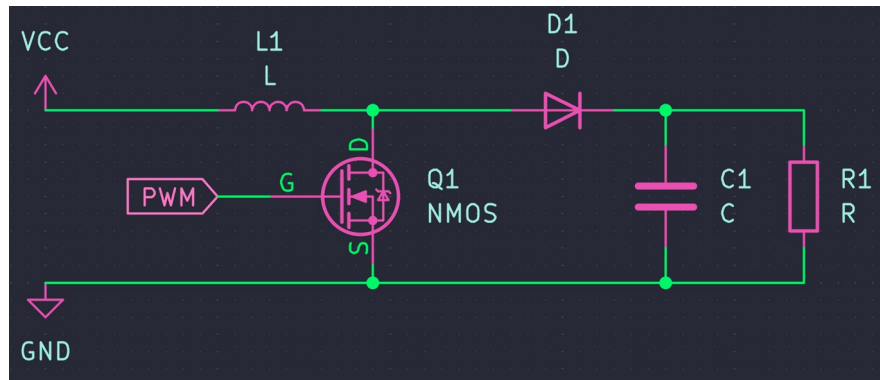
The board design is based on the UC3843 switching regulator.





Shot card V5 board's PCB and finished product

From the schematic diagram it is possible to spot the DC-DC boost



converter circuit. For clarity here is a “bare-bone” version of the board.

Kicker module demonstrating circuit

The circuit has two different states :

- **NMOS “Closed”**: the supply is basically shorted to ground, therefore the current increases linearly through the *inductance* L . This phenomenon causes the *inductor* to build up a *magnetic field* which also grows linearly. The duration of this state last **T_1**
- **NMOS “Open”**: The current that has built up in the inductor charges the *capacitor* C with a constant current. This allows the *capacitor* to charge at a higher voltage compared to the power supply one. The diode D controls the current flow to prevent a

back current to flow in the supply. The duration of this state last **T2**

→ **Equation of Step-Up circuit:** $V_o = V_i \cdot \left(1 + \frac{T_2}{T_1}\right)$

Regulating **T1** and **T2** it is possible to regulate the output voltage. The input signal of the circuit is given by a **PWM** generated signal, with variable **duty cycle**. The switching regulator is the UC3843, in the smd package.

Using the following equations we were able to calculate the output velocity of the ball should end up with, ideally.

$E = \text{energy} / C = \text{Capacitance} / V = \text{Voltage}$

$$E = \frac{1}{2} \cdot C \cdot V^2$$

Ideally the conversion of energy from potential energy, in the capacitor, into kinetic energy in the ball should be perfect, thus the velocity of the ball should be defined as:

$$v = \sqrt{\frac{2E}{m}} \quad v = \text{speed}$$

Since the rules limit the kicker voltage at **48 V** we had to increase the capacitance of the overall system, that's why we've increased the capacitors to **15000 μF** .

3.1.2. Main Board

The Main board is composed of two main parts:

- The Logic board
- The Power board

The logic board is the heart of our robots. It houses most of the vital components of the project. Here we can find both the two *microchip* © 's **MCUs**. The primary controller runs at a frequency of **48 MHz** while the secondary one runs at **4 MHz**. The board also hosts serial ports for the **Bluetooth** board and another auxiliary port. Last but not least there are two **I2C** channels. One hosted by the **secondary MCU** whose purpose is to display, on an **OLED**, the state of each line sensor, as well as calibration uses. While the other one has as its master the **main microcontroller**. This channel is populated by another **OLED Display** and by the **BNO055 IMU**. Lastly each **MCU** has an **ICSP** connector and switch, used for programming. If we sum the number of pins available on the board, coming from both **MCUs**, the total amounts to almost 54 GPIO, who are all populated. This allows us to have a total control of all the hardware onboard.

The board is equipped with some additional circuitry such as the ultrasonic sensors attachments, debug and strategies selectors, a buzzer and line sensors state LEDs (that also have the function of communicating the state of each line sensor from the secondary MCU to the main one through a direct connection). The board is also equipped with 6 infrared sensors that allow us to know when the ball is close to the robot.

The main purpose of the Power board is to control each of the four motors used for omnidirectional movement. The board presents two **ST L6207N H bridges**. Each IC can control the direction and speed of two motors. They require some external circuitry to work properly and to protect both the IC and the MCU in case of short circuit or overheat.

Lastly the board hosts both the 5 V regulators. For the regulator we opted for switching ones. We calculated the power loss on the old 5 V regulator (**L7805**) when supplied with 12 V . Knowing that our logic circuitry draws around 200 mA it's easy to find the power dissipated by the IC:

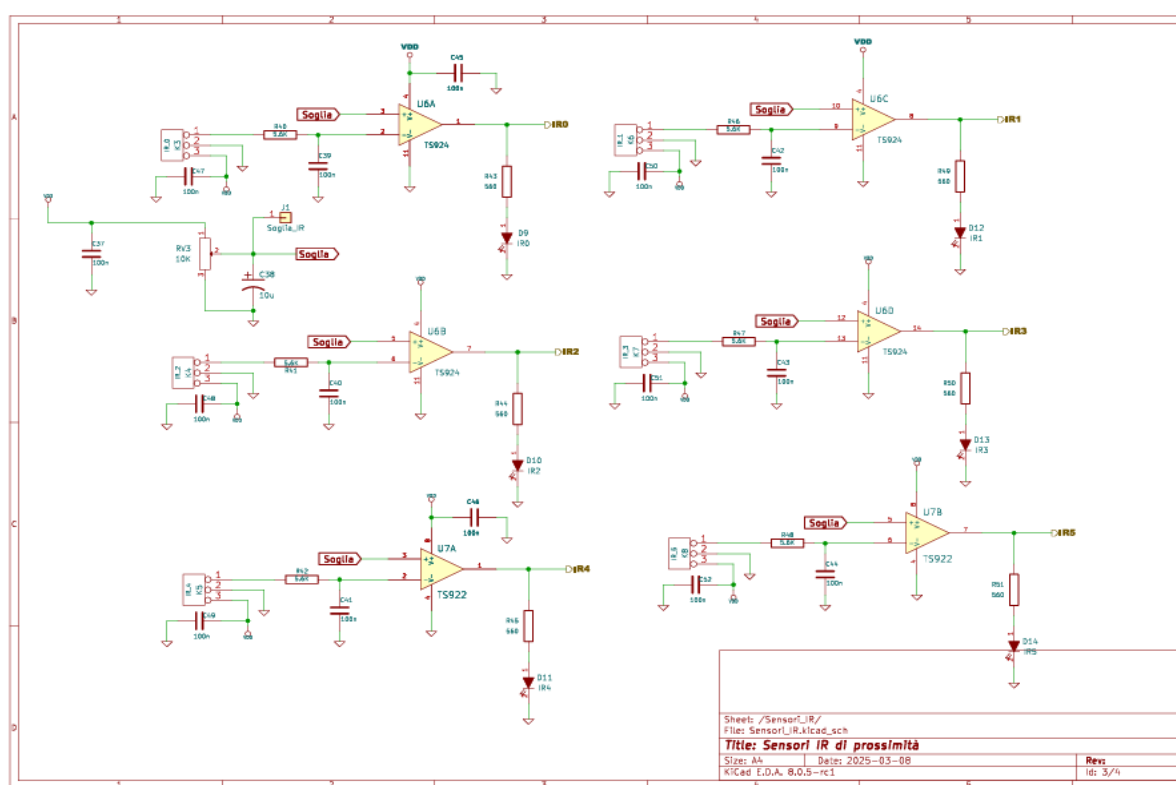
$$PD = (V_{CC} - V_{regulator}) \cdot I_{logic} = (12 - 5) \cdot 200 \cdot 10^{-3} = 1.4\text{ W}$$

Which is a lot. Not to mention that there is a power loss of almost 60% and using this IC means that it would be mandatory to provide it with a beefy heatsink since from the datasheet it specifies that the temperature of the component rises 60° at every watt the component dissipates.

The overall working principle of a switching IC is the same used in the kicker step-up circuit.



Secondary MCU Schematic







3.2. Sensors and measuring units

Viper S and **Viper S²** have plenty of sensors scattered all over their surface. They help us develop and perform actions that require precise movements. The most important measuring units are the following.

3.2.1 Main MCU

The main MCU has to manage the data that it receives from different devices and with different protocols.

In case the ball or the goals aren't found all their respective values will be saved as "-1", used as a "not found" flag in the performance conditions.

The second highest priority communication is the one with the secondary **MCU**. It calls a low priority interrupt which pauses all operations of the main program. Here the line sensor's status is copied by the main controller. Once that is done the interrupt routine ends but a flag in the performance process is raised. This flag activates the "reentry" condition which calculates the right maneuver for avoiding the out of bounds.

```
// --- CASI COMBINATI, PRIORITA' ALTA ---
if ((LIN_AV_TEMP == 1 || LIN_AV_DX_TEMP == 1) && LIN_DX_TEMP == 1) // AV e DX
{
    calcolaPwmDir(sudovest,ERR);
    goto MANOVR;
}

if ((LIN_AV_TEMP == 1 || LIN_AV_DX_TEMP == 1) && LIN_SX_TEMP == 1) // AV e SX
{
    calcolaPwmDir(sudest,ERR);
    goto MANOVR;
}

if (LIN_IND_TEMP == 1 && LIN_DX_TEMP == 1) // IND e DX
{
    calcolaPwmDir(nordovest,ERR);
    goto MANOVR;
}

if (LIN_IND_TEMP == 1 && LIN_SX_TEMP == 1) // IND e SX
{
    calcolaPwmDir(nordest,ERR);
    goto MANOVR;
}

if (LIN_DX_TEMP == 1 && LIN_SX_TEMP == 1 && ULREAR <= 40)
{
    calcolaPwmDir(avanti,ERR);
    goto MANOVR;
}

if (LIN_DX_TEMP == 1 && LIN_SX_TEMP == 1 && ULFRONT <= 40)
{
    calcolaPwmDir(indietro,ERR);
    goto MANOVR;
}

// --- CASI SINGOLI ---
if ((LIN_AV_TEMP == 1 || LIN_AV_DX_TEMP == 1) && FLAG_DIR != 7) // sens AV, non stavo andando indietro
{
    calcolaPwmDir(indietro,ERR);
    goto MANOVR;
}

// Correzione: qui servono &&, non ||.
if (LIN_IND_TEMP == 1 && FLAG_DIR != 0 && FLAG_DIR != 1 && FLAG_DIR != 2 && FLAG_DIR != 12 && FLAG_DIR != 13)
{
    calcolaPwmDir(avanti,ERR);
    goto MANOVR;
}

if (LIN_SX_TEMP == 1 && FLAG_DIR != 5 && FLAG_DIR != 4)
{
    calcolaPwmDir(dx,ERR);
    goto MANOVR;
}

if (LIN_DX_TEMP == 1 && FLAG_DIR != 9 && FLAG_DIR != 10)
{
    calcolaPwmDir(sx,ERR);
    goto MANOVR;
}
```

```
// --- CORREZIONE LATERALE CON ULTRASUONI, BASSA PRIORITA' ---
if ((LIN_AV_TEMP == 1 || LIN_AV_DX_TEMP == 1) && ULDX < SOGLIA_PAR && ULSX > SOGLIA_CAMPO)
{calcolaPwmDir(sx,ERR);
goto MANOVR;}

if ((LIN_AV_TEMP == 1 || LIN_AV_DX_TEMP == 1) && ULSX < SOGLIA_PAR && ULDX > SOGLIA_CAMPO)
{calcolaPwmDir(dx,ERR);
goto MANOVR;}

BEEP();
BOOP(); //faccio capire che nemmeno le altre condizioni sono verificate
//Se non entro in nessuna condizione faccio switch con FLAG_DIR
switch(lastDir)
{
case 0: //AVANTI
case 1: //15°
case 2: //30°
case 12: //-30°
case 13: //-15°
calcolaPwmDir(indietro,ERR); //AVANTI();
break;
case 3: //NE
calcolaPwmDir(sudovest,ERR); //DX();
break;
case 4: //60°
case 5: //Dx
calcolaPwmDir(sx,ERR); //NORDEST();
break;
case 6: //SE
calcolaPwmDir(nordovest,ERR); //SUDOVEST();
break;
case 7: //INDIETRO
calcolaPwmDir(avanti,ERR); //NORDOVEST();
break;
case 8: //SO
calcolaPwmDir(nordest,ERR); //SUDEST();
break;
case 9: //Sx
case 10: //-60°
calcolaPwmDir(dx,ERR); //NORDOVEST();
break;
case 11: //NO
calcolaPwmDir(sudest,ERR); //SUDEST();
break;
} //SWITCH
```

This algorithm determines the angle at which the robot crossed the white line by evaluating the known positions of the line sensors. When one or more sensors are triggered, the system analyzes the activated sensor combinations and selects the most appropriate escape maneuver.

In cases where multiple sensors are active simultaneously, priority is given to specific combined conditions (e.g., front + side sensors). If no clear combined condition is met, the system falls back to the opposite direction of the last performed maneuver (lastDir) to ensure the robot returns inbound.

The ultrasonic sensors are read directly through analog pins of the **MCU**. To reduce noise and improve measurement stability, each sensor is sampled multiple times and the **arithmetic mean** is calculated. The resulting filtered values (ULDX, ULSX, ULFRONT, ULREAR) are then used to modulate the escape strategy and avoid obstacles during the recovery maneuver.

3.2.2 IMU

The Inertial Measurement Unit is a key component for the implementation of movement in general. Thanks to this device we can measure the robot's movement in 9 degrees of freedom. Inside the IMU there are 3 smaller sensors: a gyroscope, an accelerometer and lastly a magnetometer. Each of these sub components measure the change of position in 3 degrees of freedom. On a rover like our robots some of these measurements are not important for the overall result. Also the IMU is a big upgrade compared to the magnetometer used on our robots many iterations prior, since this sensor is faster and less subject to noise caused by external devices.

On our robots we use the **BN0055** which gives us the angle measured from the initial "north" position to the actual orientation of the robot. The initial north position is set when the cycle starts, pointing at the opponent goal. The feedback given by this peripheral is "fed" into an algorithm which uses a PID controller whose purpose is to compensate for any misalignment errors. The PID action is passed to the PWMs equations which control the motion of the system.

IMU Reading Function

```
void leggiBusCalcolaErr(void){
    //appBus = BUS;
    int i;

    CMPS03(); // restituisce il dato in BUS 0-360

    if (BUS > (BUSINI+TORX)){BUS = BUS-(BUSINI+TORX);} //tolgo l'offset iniziale
    //aggiungo torsione
    else { BUS = 360 - (BUSINI+TORX-BUS) ;} //nei due casi

    //calcolo l'errore della bussola
    if (BUS<=180) //errore positivo, rallento a SX, accelero a DX
    {ERR = BUS;}
    else
    {ERR = BUS-360;}; //ricavo l'errore negativo complemento a 255

    if ((ERR<5 && ERR>-5)) iErr = 0; //||DIR!=0//calcolo la parte integrale dell'errore
    else iErr = 0; //IERR = IERR + 0.0005*ERR

    ERR = ERR + iErr; //disattivo momentaneamente l'integrale

    //if (ERR > 180) { ERR -= 360; }
    //if (ERR < -180) { ERR += 360; }
    //debug errore
    /*
    if (i>50) {
        debugDato(ERR);
        i=0;
    }
    else i++;
    */
}
```

```
void CENTRA_PID(void)
{
  int PWM=0;
  int PID=0;
  float pp=0;
  float ii=0;
  float dd=0;
  int lasterr=0;
  float kp=1.5;
  float ki=0.05;
  float kd=6.0;

  CMPS03(); // restituisce il dato in BUS 0-255

  if (BUS >BUSINI){BUS = BUS-BUSINI ;} //tolgo l'offset iniziale
  else { BUS = 255 - (BUSINI-BUS) ;} //nei due casi

  if (BUS>TOLBUS_P && BUS<TOLBUS_N)
  {PORTD = 0;
  goto FINE_BUS_PID;} // se sono nei limiti mi fermo

  if (BUS<128) //errore positivo, rallento a SX, accelero a DX
  {LOCERR = BUS;}
  else
  {LOCERR = BUS-255;}; //ricavo l'errore negativo complemento a 255

  pp=kp*LOCERR;
  ii=i+(LOCERR*ki*0.005);
  dd=(lasterr - LOCERR)*kd*0.005;

  lasterr=LOCERR;

  PID=pp+ii+dd;

  Delayms(5); //5ms@16MHz

  if(PID<0)
  {
    PID=PID*-1;
  }

  PWM=150+PID;
```

```

if (PWM>1023)
{
    PWM=1023;
}

if (PWM<0)
{
    PWM=0;
}

if (LOCERR<2 && LOCERR>-2)
{
    PORTD = 0;
}
else
{
    if (BUS>128) //se maggiore di 180 vado a DX
    {
        PORTD = DES_PLUS;

        SetDCPWM5(PWM); //su 1024 M1 SX
        SetDCPWM2(PWM); //su 1024 M2 SX
        SetDCPWM3(PWM); //PWM M3 DX
        SetDCPWM4(PWM); //PWM M4 Dx
    }
    else //minore di 180
    {
        PORTD = SIN_PLUS; //se minore di 180 vado a SX

        SetDCPWM5(PWM); //su 1024 M1 SX
        SetDCPWM2(PWM); //su 1024 M2 SX
        SetDCPWM3(PWM); //PWM M3 DX
        SetDCPWM4(PWM); //PWM M4 Dx
    }
}

FINE_BUS_PID:
;

} //CENTRA

```

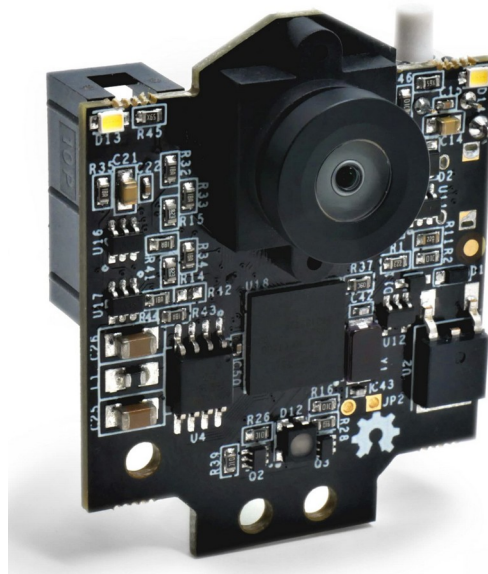
PID Calculation Function

3.2.3 Pixy 2 Camera

For the camera, we decided to use the Pixy 2 CMUcam5 Image Sensor. This powerful module guarantees a stable frame rate and features a fast onboard processor that handles all the image recognition algorithms

directly. We mounted the camera on the robot facing forward. The camera communicates with the logic board via a serial port.

Due to its mounting position, the PixyCam provides a frontal field of view. Although it typically faces only one goal at a time, the color signatures for both goals are stored in its memory; using the PixyMon software, we can train the camera to recognize multiple color signatures. The camera communicates with the main microcontroller via the I2C protocol, transmitting data on which signature is detected and the size of the recognized blob. Through the control logic in the main code, this real-time data allows us to evaluate whether we have a clear shot, need to adjust our alignment to center the goal, or if an opponent is obstructing the line of sight. Furthermore, a hardware switch on the PCB allows us to quickly select the target goal color before the match.



Pixy 2 on Viper S and Viper S²

3.2.4 Ultrasonic sensors

The **ultrasonic sensors** are among the first distance measurement units available on our robots. Their fast response time and adequate precision make them suitable for quick obstacle detection during the match. Since they provide an analog output, the measured signal can be affected by electrical noise that may reduce the reliability of the acquired data. The sensor used in this project is the DFRobot SEN0307 waterproof ultrasonic sensor.

For the hardware solution, we connected a 10 μF capacitor between the sensor output and ground. This helped filtering a large part of the signal fluctuations and improved the overall stability of the measurements. We then connected the sensor to an oscilloscope to compare the signal before and after the filtering circuit.



Multi-purpose Integration of Ultrasonic Sensors

Ultrasonic sensors are employed across various core functionalities within the robot's control system. Specifically, the data acquired from these transducers is utilized for:

- **The Torsion Maneuver:** As detailed previously, to provide the spatial data necessary for dynamically modulating the rotation angle.

- **Shooting Condition Assessment:** To evaluate spatial parameters and validate the optimal conditions required to trigger the kicking mechanism.
- **Field Re-entry Procedures:** To execute automated recovery routines, guiding the robot back into the active play area whenever a boundary line is detected.

```

if (LIN_IND_TEMP == 1 && LIN_SX_TEMP == 1) // IND e SX
{calcolaPwmDir(nordest,ERR);
 goto MANOVR;}

if (LIN_DX_TEMP == 1 && LIN_SX_TEMP == 1 && ULREAR <= 40)
{calcolaPwmDir(avanti,ERR);
 goto MANOVR;}

if (LIN_DX_TEMP == 1 && LIN_SX_TEMP == 1 && ULFRONT <= 40)
{calcolaPwmDir(indietro,ERR);
 goto MANOVR;}

// --- CASI SINGOLI ---
if ((LIN_AV_TEMP == 1 || LIN_AV_DX_TEMP == 1) && FLAG_DIR != 7) // sens AV, non stavo andando indietro
{calcolaPwmDir(indietro,ERR);
 goto MANOVR;}

// Correzione: qui servono &&, non ||.
if (LIN_IND_TEMP == 1 && FLAG_DIR != 0 && FLAG_DIR != 1 && FLAG_DIR != 2 && FLAG_DIR != 12 && FLAG_DIR != 13)
{calcolaPwmDir(avanti,ERR);
 goto MANOVR;}

if (LIN_SX_TEMP == 1 && FLAG_DIR != 5 && FLAG_DIR != 4)
{calcolaPwmDir(dx,ERR);
 goto MANOVR;}

if (LIN_DX_TEMP == 1 && FLAG_DIR != 9 && FLAG_DIR != 10)
{calcolaPwmDir(sx,ERR);
 goto MANOVR;}

// --- CORREZIONE LATERALE CON ULTRASUONI, BASSA PRIORITA' ---
if ((LIN_AV_TEMP == 1 || LIN_AV_DX_TEMP == 1) && ULDX < SOGLIA_PAR && ULSX > SOGLIA_CAMPO)
{calcolaPwmDir(sx,ERR);
 goto MANOVR;}

if ((LIN_AV_TEMP == 1 || LIN_AV_DX_TEMP == 1) && ULSX < SOGLIA_PAR && ULDX > SOGLIA_CAMPO)
{calcolaPwmDir(dx,ERR);
 goto MANOVR;}

```

Interrupt

```

if(TOCCOPIN == 0){
    ContSparo = 0;
    TOCCO = 0;
}
else{ContSparo++;}
if(ContSparo >= 10){
    TOCCO = 1;
}

//verifico le condizioni di sparo se ho il tocco attivo
//LEGGI_ULTRA();

if((TOCCO==1 && DIR_PORTA==8 && ((SIGL==FIRMAGIALLO && SEL_AUX ==0)|| (SIGL==FIRMABLU && SEL_AUX ==1))) || (TOCCO==1 && (ULDX > 50 && ULSX > 50))) {

// if((BUS<TOLBUS_P || BUS>TOLBUS_N) && conteggiosparo>500 && ULFRONT<SOGLIA_UL_AV )
// {
    CONT=0;
    VEL=1.5;
    K_BUS_H=4; //3
    while (CONT<100)
    {
        leggiBusCalcolaErr(); // restituisce il dato in BUS 0-255
        AVANTI(); // calcolaPwmDir(avanti,ERR);
        CONT++;
    };

    LANCIO =1;
    Delays(TEMPO_SPARO); //50 ms@16MHz,
    LANCIO = 0;
    TOCCO = 0;
    PORTD = 0;
    conteggiosparo=0;

    Delays(250);
}
}

```

Shooting condition with ultrasonic sensor

3.2.5 Ball Holder Sensor

The ball holder sensor is the part of the robot that detects when the ball is in his mouth. It is a very simple but also very important sensor. Without it we can't shoot due to the conditions mentioned above. The hardware is composed of a very simple switch that was used in some old printers to detect the passage of a sheet of paper.

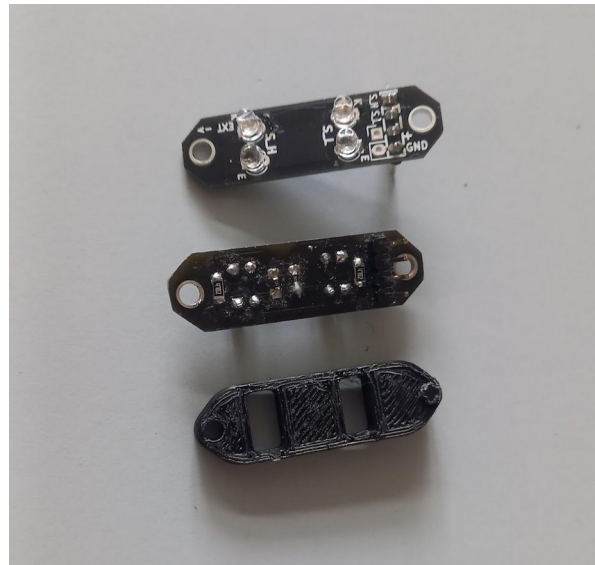


A picture of Ball holder sensor

3.2.6 Custom line sensors

Both **Viper S** and **S²** are equipped with 10 line sensors. The sensors are positioned, in pairs, in a pentagon shape having the right, rear and left sensors form a cross, while the last two pairs are positioned at an angle of 15 and -15 compared to the robot forward center. Each vertex of the pentagon has an internal and external sensor. With this configuration we can firstly identify the presence of the line using the external sensors, then once the robot gets too out of bounds and triggers an internal sensor a reentry maneuver will be performed.

Each line sensor consists of a small PCB with 2 pairs formed by an LED and a phototransistor one next to the other. The light difference between the field and the line will cause the phototransistor to change his voltage value. This analog value is read for each pair by the secondary **MCU** using an analog pin.



Line sensor board near the 3D printed support

All the line sensors are mounted on custom-made, 3D-printed supports bolted to the bottom frame of the robot. These supports are specifically designed to prevent unwanted optical interference-effectively shielding the phototransistors from any direct, undesired propagation of the light emitted by the red LEDs.



the 3D printed support for interference

3.2.7 Secondary MCU

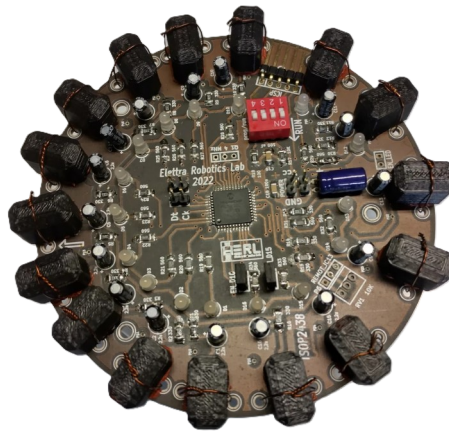
The secondary **MCU** is tasked with reading the 10 analog line sensors. Its code is written entirely in assembly language. This controller also drives an OLED **Display** which is used for debug and tuning purposes. The processor has a tuning algorithm which generates the thresholds of activation for the lines.

3.2.8 Custom Ball Detection Board

This board is a specialized, 360-degree infrared sensor designed for competitive robotics, such as RoboCup Junior, to track IR-emitting balls without any blind spots. The circular PCB features an outer ring of sixteen TSOP2438 infrared receivers, which are specifically tuned to the 38 kHz frequency emitted by official competition balls.

In the center of the board, a surface-mount microcontroller continuously processes the incoming sensor data. It communicates the ball's relative position to the robot's main controller using the I2C protocol, which is clearly indicated by the SDA and SCL pins. Because it uses I2C, the board includes a prominent four-position red DIP switch, allowing users to easily configure its hardware address so multiple devices can share the same communication bus without interference.

Finally, the power delivery system is designed for stability. Powered via standard VCC and GND pins, the board utilizes large electrolytic capacitors to prevent voltage drops caused by the robot's motors, alongside smaller ceramic capacitors that filter out high-frequency electrical noise, ensuring reliable and accurate sensor readings.



Custom Ball detection board

3.2.9 Ball following algorithm

To ensure the ball-tracking algorithm functioned effectively, we mapped out a specific set of directional maneuvers for the robot. Achieving the high level of precision required by the code's logic—both for seeking the ball and avoiding obstacles—Our solution to achieve the most precise movements possible is based on the **PWM value**, which is calculated specifically for each motor and direction. The robot supports a total of **14 distinct movement directions**.

To compute them, we developed a dedicated **Excel spreadsheet** which, given an offset and a normalization factor as inputs, returns the PWM values for all four motors.

Fattore di normalizzazione	350	offset	0	Offset fisso 130
----------------------------	-----	--------	---	------------------

		pwm5 SUL PIC18F66K80			
		pwm1	pwm2	pwm3	pwm4
	Max				
0	0,70	302	350	-302	-350
15	0,88	312	188	-152	-349
30	1,00	320	75	-46	-349
45	1,05	326	-19	42	-349
60	0,98	350	-117	134	-367
90	0,78	350	-350	350	-350
135	1,05	42	-349	326	-19
180	0,70	-302	-350	302	350
225	1,05	-326	19	-42	349
270	0,78	-350	350	-350	350
300	1,03	-127	349	-332	111
315	1,05	-42	349	-326	19
330	1,03	45	339	-310	-73
345	0,88	152	349	-312	-188

PWMs calculation workflow in Excel

```
// DIREZIONI
switch(DIR)
{
    case 0:{
        NOPALLA++;
        if(NOPALLA >= 200){
            VEL = VEL - 0.2;
            if (ULREAR <= 50) {calcolaPwmDir(avanti,ERR);}
            else{calcolaPwmDir(indietro,ERR);}; //accentro avanti e indietro

            if (ULDX < SOGLIA_CENTRO && ULSX > SOGLIA_CAMPO) {
                calcolaPwmDir(sx,ERR);
            }
            if (ULSX < SOGLIA_CENTRO && ULDX > SOGLIA_CAMPO) {
                calcolaPwmDir(dx,ERR);
            }
        }
        break;
    }
    case 5://avanti
    {
        if((IR0==1||TOCCO==1)&&((SIGL==FIRMAGIALLO && SEL_AUX ==0)|| (SIGL==FIRMABLU && SEL_AUX ==1)))
        {
            if(DIR_PORTA==8)calcolaPwmDir(avanti,ERR);
            else if (DIR_PORTA>8)calcolaPwmDir(trenta,ERR);
            else calcolaPwmDir(menotrenta,ERR);
        }
        else calcolaPwmDir(avanti,ERR);
        break;
    }
    case 4:
    {
        if(IR0 == 1&&TOCCO ==0){calcolaPwmDir(sx,ERR);}; //aggiramento senza tocco

        if((TOCCO==1)&&((SIGL==FIRMAGIALLO && SEL_AUX ==0)|| (SIGL==FIRMABLU && SEL_AUX ==1)))
        {
            if(DIR_PORTA==8)calcolaPwmDir(avanti,ERR);
            else if (DIR_PORTA>8)calcolaPwmDir(trenta,ERR);
            else calcolaPwmDir(menotrenta,ERR);
        }
        else calcolaPwmDir(avanti,ERR);
        break;
    }
}
```

Ball tracking code

4. Mechanical structure and hardware

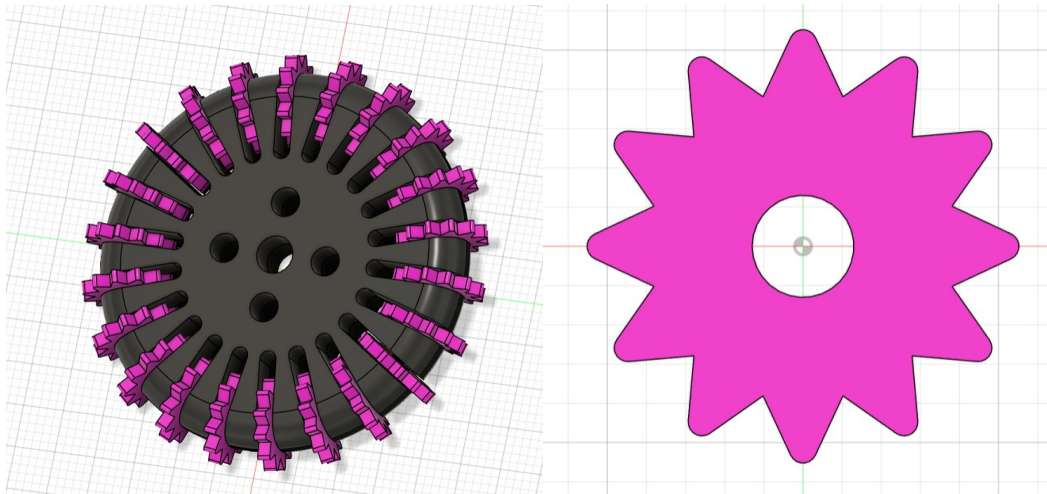
The main goal for mechanical improvements this year was to decrease the height of our robots compared to the previous ones, in order to enhance stability and movements. To make this possible we had to reduce the

distance between the two main frames, which is not an easy task due to the presence of the kicker's structure. Another task for this year mechanics was to change the angulation of the wheels for a better performance, and also produce a new kind of omni wheel with enhanced grip.

4.1. Custom omnidirectional wheels

To improve the grip of our wheels the designers landed on a tooth-shaped washer which will replace the old smooth washers we used. There we concluded that the new wheels helped with both the responsiveness of the movements and also with the head-to-head challenges when 2 robots would run into each other.

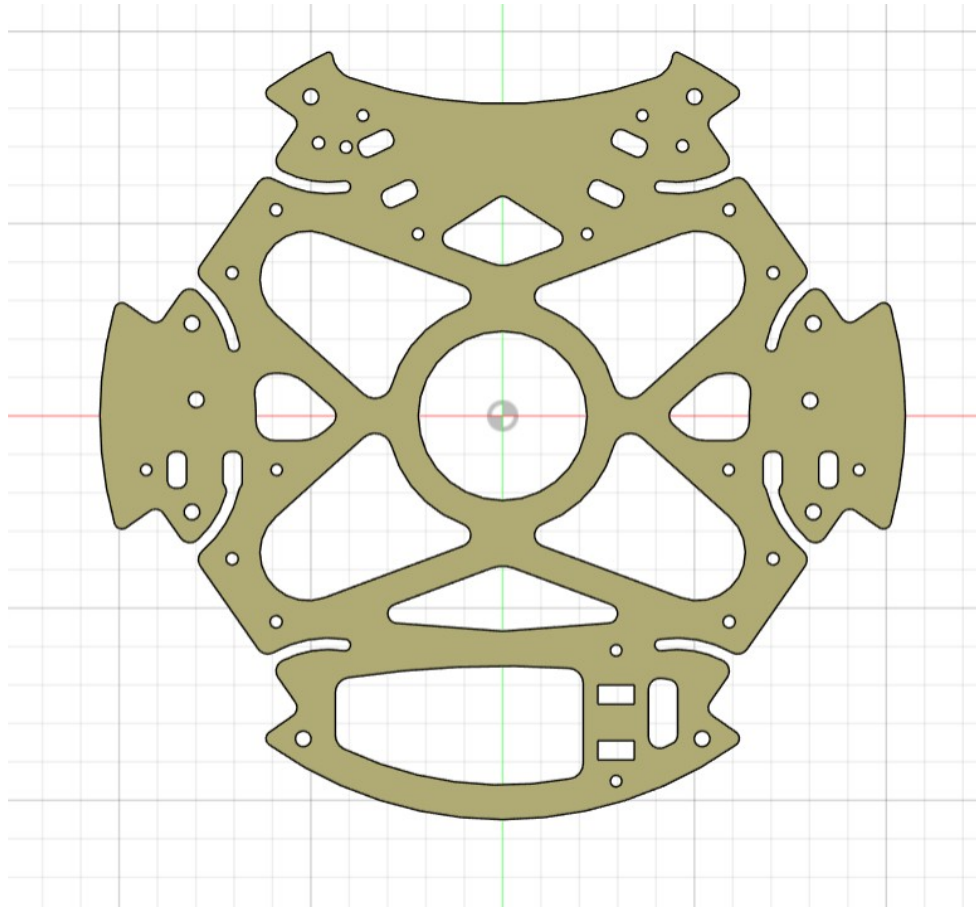
To avoid damaging the field the washers are 3D printed.



Omnidirectional wheel with smooth washers & tooth-shaped washer

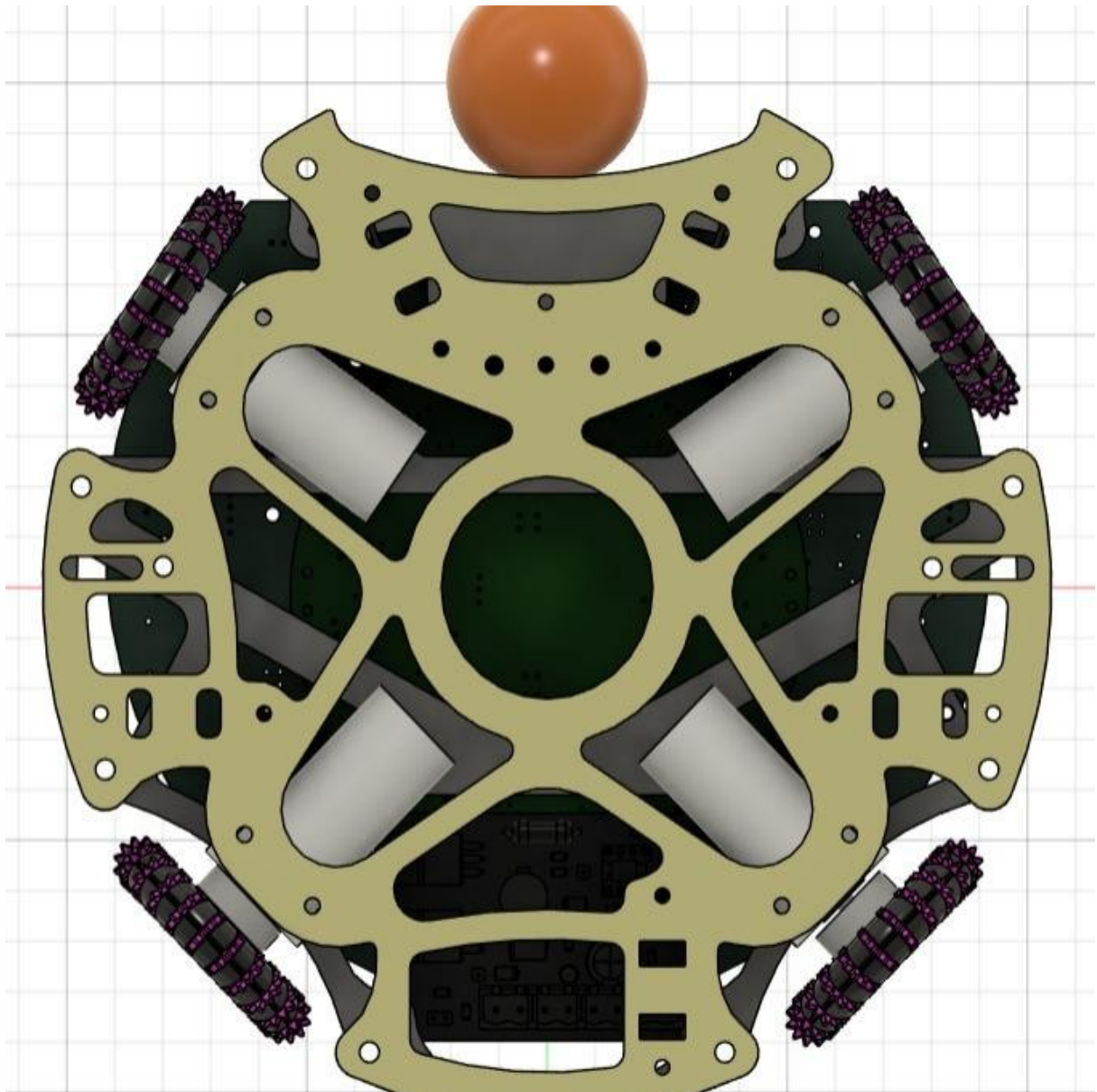
4.2. Chassis

In the design of the frame we kept 4 motors with an asymmetric configuration, to give more power to the robot, the distance between the wheels increases to give more stability. The kicker coil is fixed this time on the upper frame to interfere less with the motors. The line sensors maintain a pentagonal arrangement with two phototransistors for each block. This should allow us to obtain the crossing direction of the white line.



The lower frame design, with curved suspensions

to improve the robot's grip we designed curved suspensions, thus exploiting the elasticity of the lower polycarbonate frame. In order to protect the wheels, we also designed bumpers, as we can see in the image below.

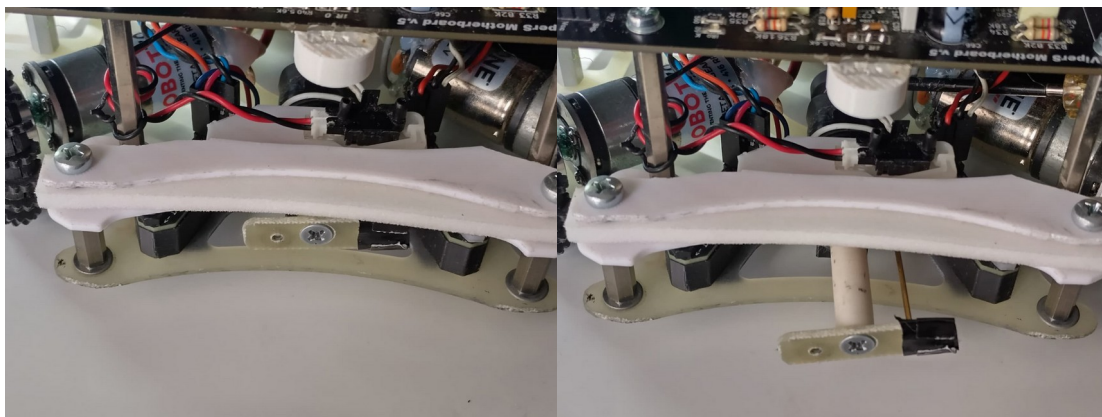


Bottom view of the robot. You can see the bumpers and the arrangement of the line sensors.

4.3. Mounth

In addition we put in front of the ball holder sensor, a little neoprene strip to protect it because it's very fragile and it's very easy to break. The structure of the neoprene stripe is composed by two screws that hold in place the stripe with two little plastic spaces between them to reduce friction and avoid the breakage. To increase the smoothness of the moving of the stripe we also added to washers that act like bearing and help to reduce the friction. With that solution we noticed that it can easily detect the ball also when it's on the edge of the mouth with a very small quantity of force needed.

Furthermore, testing revealed that this neoprene strip enhances ball control by dampening the impact force of the ball.



Ball holder sensor protected with neoprene.

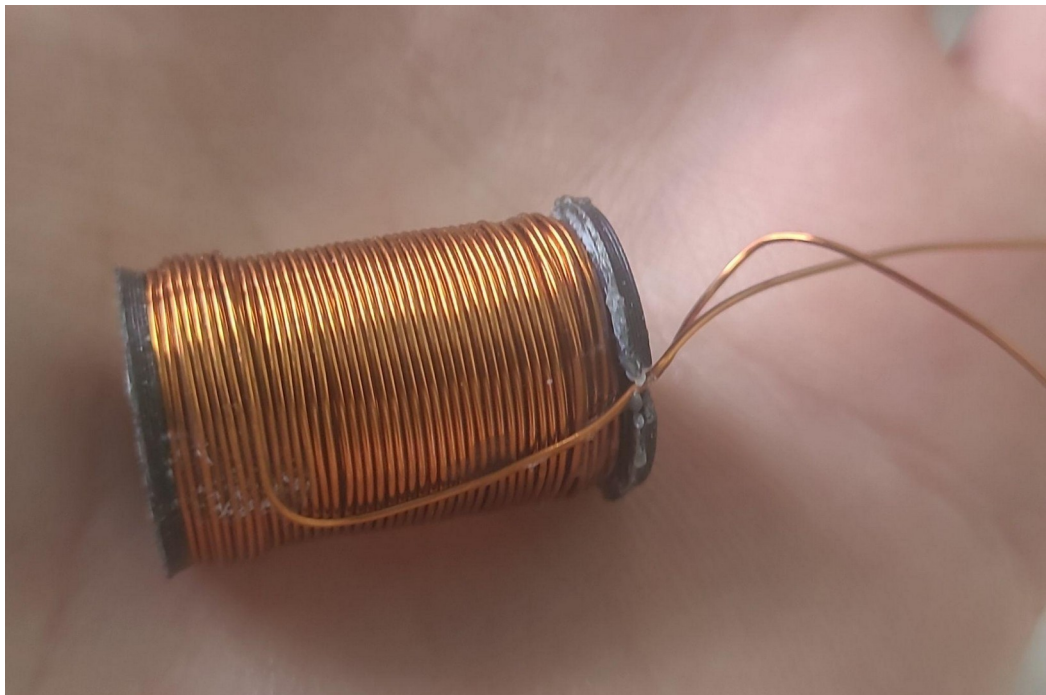
4.4. kicker

The Kicker was made with a homemade coil, the piston is made half in iron and half in non magnetic material, the iron part is positioned external and kept in pose by a spring. When current flows through the coil, the iron portion is pushed inside the coil, so that the piston can kick the ball. The force of the piston is due basically by two factors: current and magnetic

field. In order to increase the power of the kicker we decided to reduce the resistance of the coil, from 0.7Ω . So when the MosFet of the kicker module is driven On by the **MCU**, the burst current is: $\frac{48V}{0.7\Omega} = 69A$. This means that we have to use an appropriate MosFet.

$$\tau = r \cdot c = 0.7 \Omega \cdot 15 mF = 10,5 mS$$

$$T_s \leq 5 \cdot \tau \leq 5 \cdot 10,5 mS \leq 52,5 mS$$



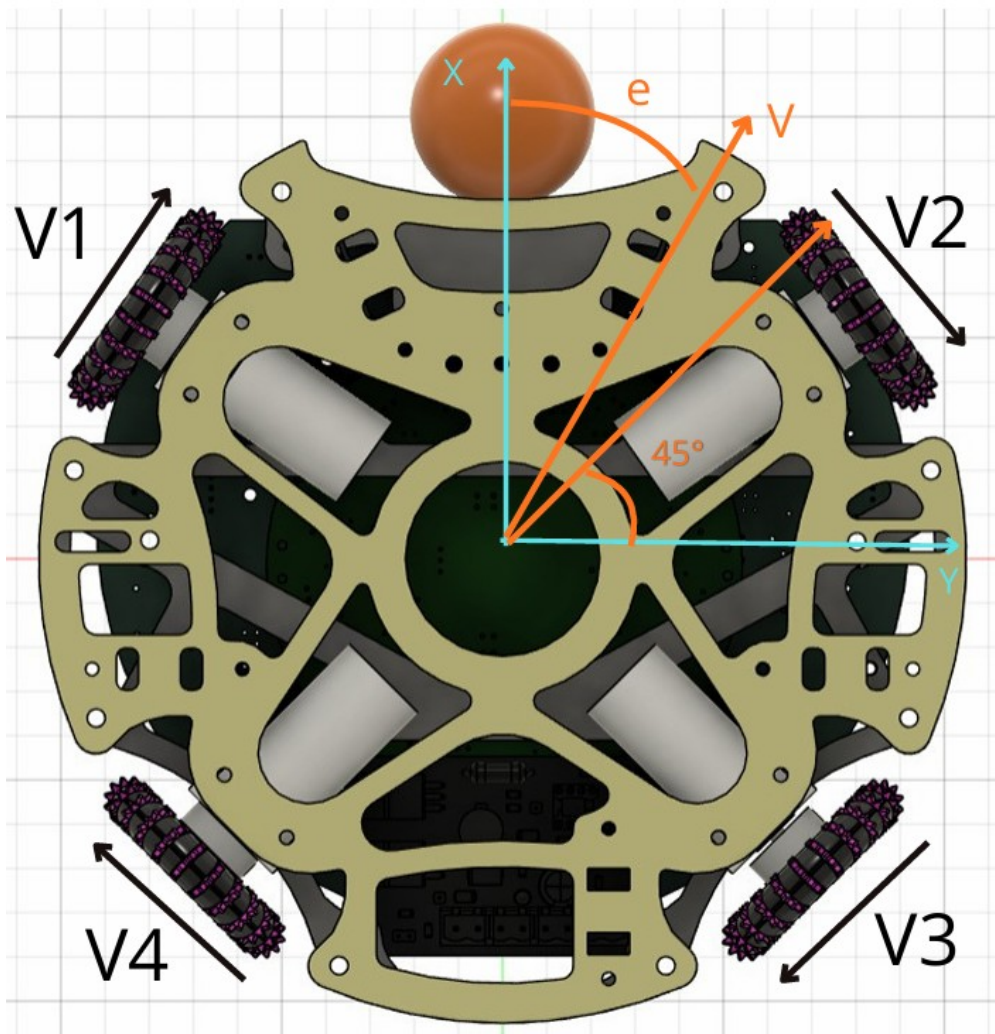
Our homemade coil. We used a 0.4mm diameter copper wire

4.5. Omnidirectional movement

The omnidirectional movement is obtained by composing the four speed vectors of the motors. The robot features an **asymmetric wheel layout**: wheels V1 and V2 form an angle of **55°** with the X-axis, while wheels V3 and V4 form an angle of **45°** with the X-axis.

The trigonometric values used are:

$$\begin{aligned}\sin(55^\circ) &\approx 0.8192 & \cos(55^\circ) &\approx 0.5736 \\ \sin(45^\circ) &\approx 0.7071 & \cos(45^\circ) &\approx 0.7071\end{aligned}$$



By decomposing the four vectors along the two axes, we obtain the following **forward kinematics** equations:

$$V_x = (V_1 \cdot \cos 55^\circ - V_2 \cdot \cos 55^\circ - V_3 \cdot \cos 45^\circ + V_4 \cdot \cos 45^\circ)$$

$$V_y = (V_1 \cdot \sin 55^\circ + V_2 \cdot \sin 55^\circ - V_3 \cdot \sin 45^\circ - V_4 \cdot \sin 45^\circ)$$

These equations alone are not directly invertible (since there are 4 unknowns).

To obtain the inverse kinematics, we introduce the no-rotation constraint ($\omega = 0$):

$$V_1 + V_2 + V_3 + V_4 = 0 \quad V_1 + V_2 + V_3 + V_4 = 0 \quad V_1 + V_2 + V_3 + V_4 = 0$$

Once again the equations aren't solvable since the system has 4 unknowns. To achieve the inverse kinematics it's needed to use the matrix calculation. If we At this point, assuming that the resulting velocity V (composed of V_x and V_y) forms an angle α with the X-axis, we can write:

$$V_x = V \cdot \cos \alpha \quad V_y = V \cdot \sin \alpha$$

Using matrix calculations, the equations for each motor become:

$$V_1 = V \cdot (\cos \alpha \cdot \cos 55^\circ + \sin \alpha \cdot \sin 55^\circ) = V \cdot \cos(\alpha - 55^\circ)$$

$$V_2 = V \cdot (-\cos \alpha \cdot \cos 55^\circ + \sin \alpha \cdot \sin 55^\circ) = V \cdot \cos(\alpha - 125^\circ)$$

$$V_3 = V \cdot (-\cos \alpha \cdot \cos 45^\circ - \sin \alpha \cdot \sin 45^\circ) = V \cdot \cos(\alpha - 225^\circ)$$

$$V_4 = V \cdot (\cos \alpha \cdot \cos 45^\circ - \sin \alpha \cdot \sin 45^\circ) = V \cdot \cos(\alpha - 315^\circ)$$

The speed of each motor is directly proportional to the PWM signal applied to it.

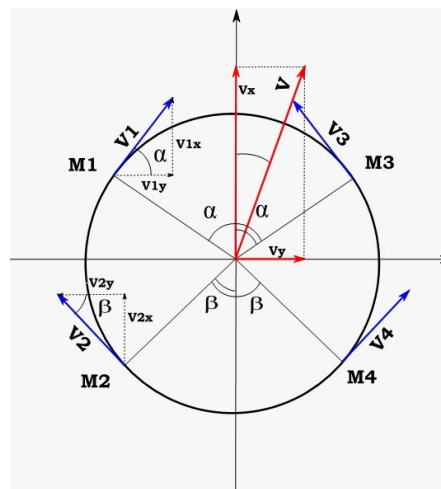
For the testing phase, we utilized a custom debugging script specifically designed to isolate and evaluate the robot's kinematics. By calculating the theoretical point of impact against a boundary wall for every directional vector, we were able to determine that the standard formulas exhibited significant inaccuracies.

Consequently, we fully adopted the newly developed experimental equations. This transition yielded a substantial improvement, ensuring a much higher degree of precision and reliability across all executed movements.

tabella errori pwm

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1		pwm vipers				old				test					
2	angoli	-35				-24	-57	-41	-29	-14	-24	-36	-31		-36
3	15	-35	-7	-23	-25	-95	-109	-100	-97	-72	-67	-85	-63		
4	30	-68	-53	-50	-48										
5		-68													
6	media 15	-25				-37.75				-28.66666667					
7	media 30	-56.66666667				-100.25				-71.75					
8															
9															

old pwm excel error table



New experimental model of resolution

$$V_x = (V_1 \cdot \sin 55^\circ + V_2 \cdot \sin 45^\circ + V_3 \cdot \sin 55^\circ + V_4 \cdot \sin 45^\circ)$$

$$V_y = (V_1 \cdot \cos 55^\circ - V_2 \cdot \cos 45^\circ - V_3 \cdot \cos 55^\circ + V_4 \cdot \cos 45^\circ)$$

$$0=V_1+V_2-V_3-V_4$$

$$V_1 = \frac{\sin 55^\circ}{2(\sin^2 55^\circ + \sin^2 45^\circ)} V_x + \frac{1}{2(\cos 55^\circ + \cos 45^\circ)} V_y \quad \square$$

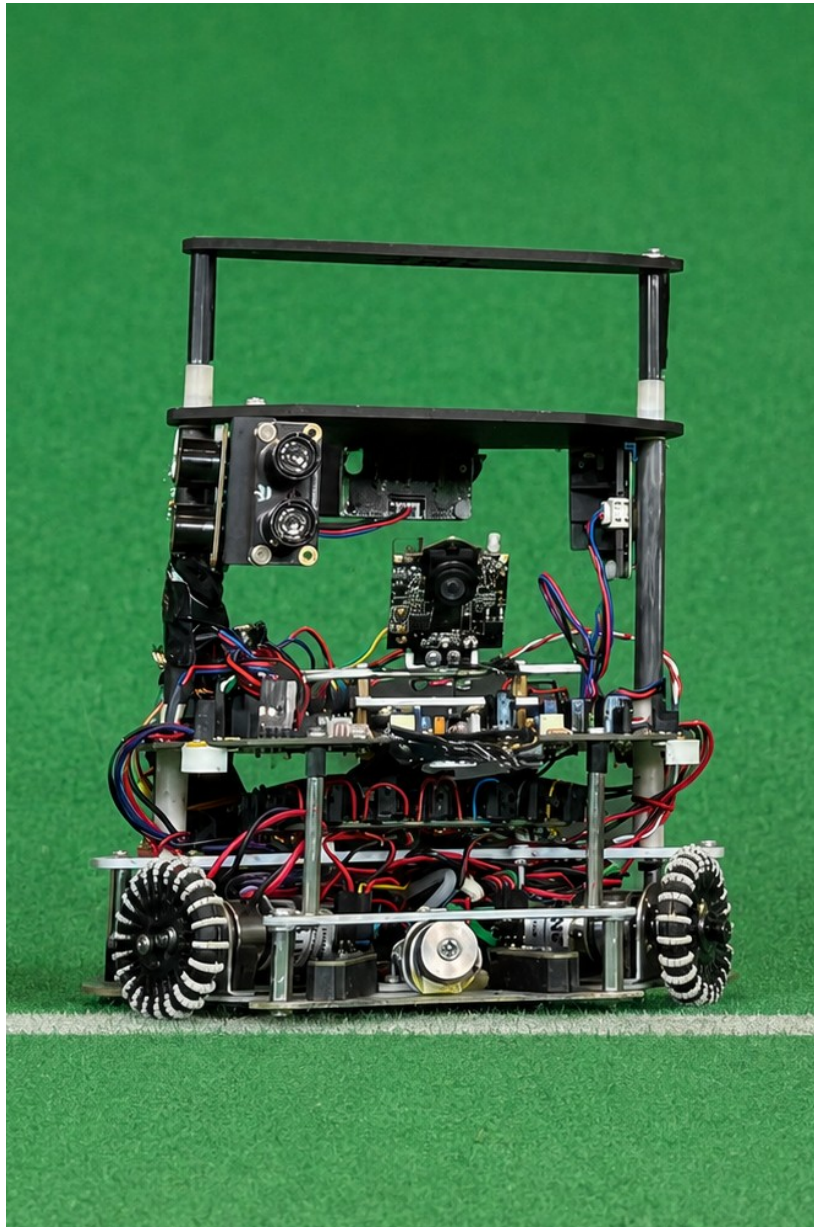
$$V_2 = \frac{\sin 45^\circ}{2(\sin^2 55^\circ + \sin^2 45^\circ)} V_x - \frac{1}{2(\cos 55^\circ + \cos 45^\circ)} V_y \quad \square$$

$$V_3 = \frac{\sin 55^\circ}{2(\sin^2 55^\circ + \sin^2 45^\circ)} V_x - \frac{1}{2(\cos 55^\circ + \cos 45^\circ)} V_y \quad \square$$

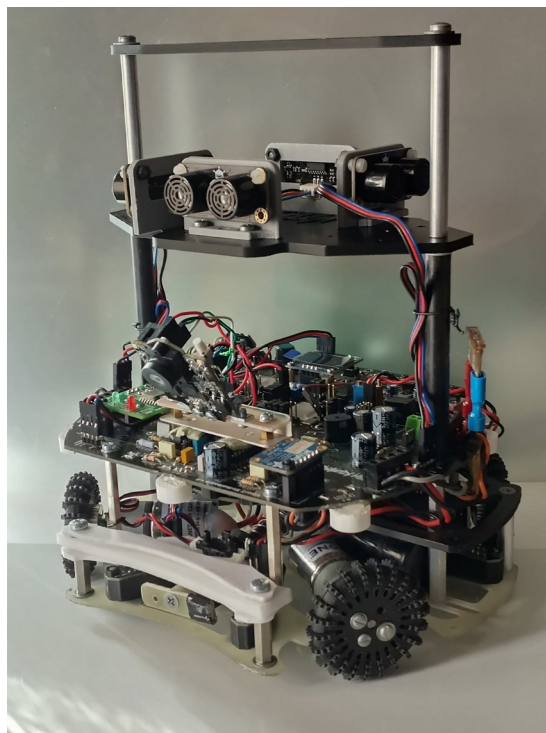
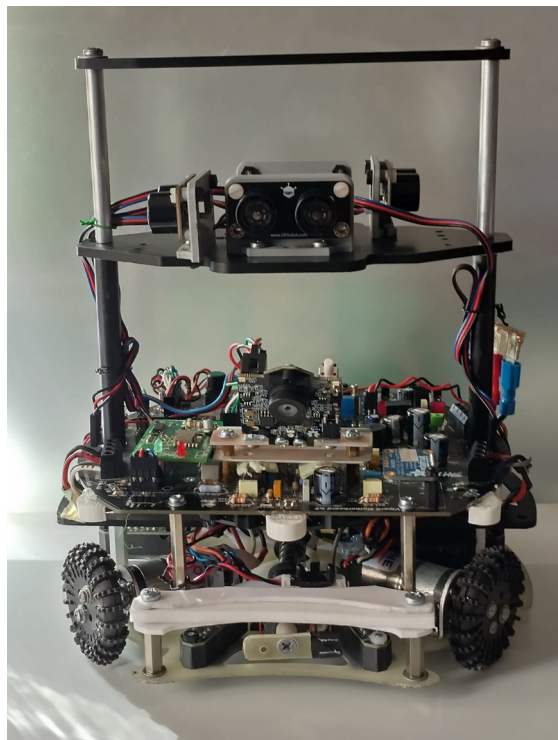
$$V_4 = \frac{\sin 45^\circ}{2(\sin^2 55^\circ + \sin^2 45^\circ)} V_x - \frac{1}{2(\cos 55^\circ + \cos 45^\circ)} V_y \quad \square$$

New experimental formulas

5. Gallery



Robot Viper S on the field at the Italian tournament



Viper S²